



Programação WEB Avançada

Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

3

Arquitetura com Application Factory e Contexto no Flask

Resumo

Nesta unidade aprofundamos a arquitetura de aplicações Flask, dando continuidade à organização modular introduzida anteriormente. O foco está na compreensão dos principais problemas estruturais de projetos web em crescimento, especialmente importações circulares, acoplamento excessivo e má gestão do ciclo de vida da aplicação. A partir disso, apresentamos o padrão *Application Factory*, o uso correto de extensões desacopladas, e os conceitos fundamentais de *Application Context* e *Request Context*. Também discutimos a organização em pacotes, inicialização preguiçosa (lazy initialization), e boas práticas para projetos escaláveis, testáveis e manuteníveis. Ao final, o estudante será capaz de estruturar aplicações Flask profissionais seguindo princípios de engenharia de software e arquitetura limpa.

“Arquitetura de software é sobre tomar decisões que permitam evolução sem colapso.”

Adaptado de Martin Fowler

Problemas Estruturais em Aplicações Flask

À medida que aplicações web evoluem, a estrutura inicial — geralmente simples e funcional — tende a se tornar insuficiente. Em projetos `Flask` iniciantes, é comum concentrar toda a lógica em um único arquivo, o que funciona bem para protótipos, mas rapidamente gera dificuldades de manutenção, escalabilidade e testabilidade.

Dois problemas principais surgem com frequência:

- Importações circulares entre módulos;
- Acoplamento excessivo entre a aplicação, extensões e rotas.

Esses problemas aparecem, sobretudo, quando a aplicação cresce e passa a ser dividida em múltiplos módulos, como `models`, `routes`, `services` e `extensions`.

O Problema das Importações Circulares

Importações circulares ocorrem quando dois ou mais módulos dependem mutuamente entre si durante o carregamento do interpretador. Em `Flask`, isso acontece frequentemente quando o objeto `app` é criado em um módulo principal e importado diretamente em diversos outros módulos.

Por exemplo:

- O arquivo `app.py` cria a aplicação;
- O módulo de rotas importa `app`;
- O módulo principal importa as rotas.

Esse ciclo pode impedir a inicialização correta do projeto, resultando em erros difíceis de diagnosticar e comportamentos inesperados.

O código a seguir irá introduzir o processo de criação de um aplicativo mínimo a partir do `Flask`. Lembrem-se que a classe `Flask` é a base do nosso framework. É através dela

que criamos a **instância** do nosso app, **configuramos** a aplicação e acessamos a maioria das **funcionalidades**. O que não vem da **classe** Flask provém das **extensões** que podemos instalar e/ou desenvolver.

```
1 from flask import Flask
2 app = Flask(__name__)
3 @app.route("/")
4 def index():
5     return "Ola Mundo!"
6 @app.route("/contato")
7 def contato():
8     return "<form><input type='text'></form>"
```

Um aplicativo básico é geralmente chamado de app, e passamos a variável mágica do Python, `__name__`, para a criação da instância.

As linhas 1 e 2 do código acima, basicamente resumem conceitualmente um aplicativo Flask. O que vamos adicionar a esse aplicativo são as views. Para isso, utilizamos o decorador `@app.route`, que define a rota. Este decorador intercepta e gerencia as rotas, associando uma função que criamos à URL correspondente. No caso, a rota `/` será a URL base do nosso projeto, e a função `index` retornará "Olá Mundo!".

Essa é a nossa aplicação Flask simples. Para executá-la, abra um terminal e use o comando `flask run`. Em seguida, acesse a URL¹ `localhost:5000` no navegador, e você verá a mensagem "Olá Mundo!".

E, até aqui, nenhuma novidade!

No entanto, em um projeto real, provavelmente adicionaremos mais rotas. Além da rota principal, podemos criar uma nova rota chamada `contato`, que servirá para um formulário onde o usuário poderá enviar informações de contato. A função `contato` retornará um HTML simples.

Assim, ao acessarmos a URL `localhost:5000/contato` no navegador, seremos apresentados a um formulário com um campo de entrada. Precisamos incluir botões e outros elementos que geralmente fazem parte de um formulário. Porém, a aplicação não se limitará a esse formulário de contato.

¹Em particular, sou fã do endereço `127.0.0.1:5000` — *there's no place like this!*

Essas funções que associamos às rotas são chamadas de views porque são responsáveis por enviar informações para a camada de visualização. Diferente de outros frameworks, como Django, que seguem padrões como MVC (Model View Controller) ou MVT (Model View Template), o Flask não impõe um padrão de projeto específico. Portanto, temos a liberdade de criar nossa própria estrutura.

À medida que o projeto avança, uma prática comum é dividir a aplicação em múltiplos arquivos. Por exemplo, podemos criar um arquivo chamado `views.py` para organizar nossas views separadamente do arquivo principal `app.py`. No entanto, se você fizer isso sem as devidas importações, ao rodar o Flask, você receberá um erro informando que as URLs não foram encontradas.

Quando executamos `flask run`, o Flask procura pelo arquivo `app.py` e o executa linha a linha. Este arquivo é o ponto de entrada da aplicação. Se o `app` não estiver acessível, as views em `views.py` não serão reconhecidas.

Para resolver isso, precisamos importar as views no arquivo principal. Por exemplo, podemos usar:

Código: `app.py`

```
1 from flask import Flask
2 from views import index, contato
3
4 app = Flask(__name__)
```

No entanto, se tentarmos rodar dessa forma, **aqui surgirá o famoso erro de importação circular**. Ele ocorrerá porque, ao importar as views, o Flask tentará executar o arquivo `views.py`, que por sua vez tenta importar o `app`. Essa situação gera um ciclo onde cada arquivo depende do outro para ser inicializado, resultando em um *deadlock*.

Quando você vê uma mensagem de erro indicando que o aplicativo foi parcialmente inicializado, isso geralmente aponta para um erro de importação circular. Esse problema pode ocorrer em qualquer programa Python que tente dividir o código em múltiplos arquivos. O Python segue uma ordem de importação e, ao tentar resolver as dependências, ele pode entrar em um *loop* infinito.

Código: views.py

```

1 from app import app
2 @app.route("/")
3 def index():
4     return "Ola Mundo!"
5 @app.route("/contato")
6 def contato():
7     return "<form><input type='text'></form>"

```

Para entender melhor, vamos analisar o que acontece durante a execução:

1. **Importação do app:** o Flask inicia a execução do arquivo `app.py`, importando a classe Flask e criando a instância.
2. **Importação das Views:** ao tentar importar as views, o Flask executa o arquivo `views.py` linha a linha.
3. **Tentativa de Acesso ao app:** quando o `views.py` tenta usar o decorador `@app.route`, ele não consegue encontrar a instância `app` porque a execução ainda não retornou ao `app.py`.

A Figura (3.1) ilustra a descrição acima.

No `app.py`	E então no `views.py`
<pre> from flask import Flask from views import index app = Flask(__name__) </pre>	<pre> from app import app @app.route('/') def index(): return "Hello World" </pre>

Figura 3.1: Resumo do nosso exemplo de *Circular import*.

Essa situação também resulta numa classificação de erro conhecida como “ImportError”, indicando que o aplicativo foi parcialmente inicializado.

Para evitar isso, poderíamos pensar em usar o padrão de importação em que o `app` é criado antes das views serem importadas, garantindo que o Flask tenha o contexto necessário para associar as rotas corretamente. “Mas, aí...”

A “Gambiarra” do Import no Final do Arquivo

Diante do problema de importação circular, uma solução bastante comum — embora não seja considerada elegante — consiste em adiar a importação das views para o final do arquivo principal. Na prática, muitos desenvolvedores² movem a linha de importação para depois da criação da instância da aplicação:

Código: app.py

```
1 from flask import Flask
2 app = Flask(__name__)
3
4 from views import index, contato
```

E no arquivo de views mantemos tudo igual:

Código: views.py

```
1 from app import app
2 @app.route("/")
3 def index():
4     return "Ola Mundo!"
5 @app.route("/contato")
6 def contato():
7     return "<form><input type='text'></form>"
```

À primeira vista, essa abordagem parece resolver o problema de importação circular. E, de fato, em muitos casos ela funciona. Mas por quê? Vamos entender isso, falando sobre **a ordem de execução do sistema de imports do Python**.

O Python executa os arquivos de cima para baixo, de forma sequencial. Assim, quando o interpretador executa o arquivo `app.py`, ocorre a seguinte sequência:

1. O Python importa o módulo `flask`;
2. A instância `app = Flask(__name__)` é criada;

²Não vamos pré-julgar tais desenvolvedores. Entretanto, precisamos registrar que bons desenvolvedores Python têm conhecimento legítimo sobre a arquitetura de seus códigos e das boas práticas extensivamente recomendadas na PEP 8 – Style Guide for Python Code.

3. Somente depois disso ocorre a importação de `index` e contato do módulo `views`.

Nesse momento, quando o arquivo `views.py` é executado e tenta fazer:

```
1 from app import app
```

o objeto `app` já existe no contexto de execução, pois a instância foi criada anteriormente. Dessa forma, o ciclo de dependência não causa mais falha imediata de inicialização, permitindo que a aplicação funcione.

Por que Essa Solução Não é Recomendada? Apesar de funcional, essa técnica viola uma das principais recomendações da PEP 8, o guia oficial de estilo do Python, que orienta que as importações devem ser organizadas no topo do arquivo.

Colocar *imports* no final do arquivo ou dentro de funções:

- reduz a legibilidade do código;
- dificulta a manutenção;
- torna a estrutura do programa menos previsível;
- pode introduzir comportamentos inesperados em projetos maiores.

Em comunidades *Pythonistas*, a organização e a clareza do código são altamente valorizadas. Assim, `imports` fora do topo do arquivo costumam ser vistos como um indicativo de problema estrutural na arquitetura do projeto, e não como uma solução definitiva.

Além disso, outra prática que aparece com frequência é a realização de `imports` dentro de funções, como forma de evitar o ciclo de importação. Embora isso funcione tecnicamente, também não é considerado uma boa prática em termos de design de software.

Lazy Imports e Execução Tardia (Lazy Evaluation)

Para compreender uma solução mais elegante, é necessário entender um conceito fundamental: o sistema de `import` do Python é imediato. Ou seja, quando utilizamos:


```
1 from modulo import objeto
```

o Python tenta resolver essa importação naquele exato momento da execução.

Logo, uma forma mais adequada de lidar com dependências circulares é **adiar** a execução de determinadas partes do código. Esse comportamento é conhecido como execução preguiçosa (*lazy execution*) — também denominada posterior.

Em programação, a principal estrutura que permite **execução posterior** é o conceito de objetos *callable*. No Python, são considerados *callables*:

- Funções;
- Métodos;
- Classes;
- Qualquer objeto que possa ser invocado com **parênteses**.

Ao definirmos uma função, estamos descrevendo um comportamento que só será executado posteriormente, quando ela for chamada. Isso nos permite controlar melhor o momento em que determinadas dependências serão carregadas.

Transição para uma Solução Arquitetural Elegante — a “gambiarra” do `import` no final do arquivo funciona porque altera a ordem de execução, mas não resolve a causa raiz do problema: o acoplamento excessivo entre módulos e a inicialização rígida da aplicação.

Em projetos Flask de médio e grande porte, precisamos de uma abordagem mais limpa, escalável e alinhada com boas práticas de engenharia de software. Em vez de depender da ordem dos imports, a solução recomendada é **estruturar a aplicação de modo que sua criação ocorra de forma controlada e tardia**.

Essa abordagem é implementada por meio de um padrão arquitetural conhecido como *Application Factory*, no qual a aplicação Flask deixa de ser uma variável global e passa a ser criada por uma **função** responsável pela sua configuração e inicialização.

Então, vamos em frente com essa técnica!

Aplicando o Padrão Application Factory de Forma Simples

Até aqui, vimos que mover o import para o final do arquivo resolve o problema de importação circular, mas não é uma solução elegante. Precisamos de uma abordagem que elimine o acoplamento estrutural entre os módulos.

A proposta do Flask para resolver esse tipo de problema é o uso do padrão conhecido como *Application Factory*.

A ideia central é que, em vez de criar a aplicação diretamente no escopo global:

```
1 app = Flask(__name__)
```

vamos encapsular sua criação dentro de uma função. Essa função será responsável por construir e configurar a aplicação somente quando for chamada.

Isso altera completamente o comportamento do sistema: nada acontece no momento da importação do módulo. A aplicação só passa a existir quando a função for invocada.

Portanto, precisamos reorganizar nosso exemplo anterior em dois arquivos.

Código: app.py

```
1 import views
2 from flask import Flask
3
4 def create_app():
5     """Factory principal"""
6     app = Flask(__name__)
7     views.init_app(app)
8     return app
```

Observe alguns pontos importantes:

- não existe mais `app = Flask(__name__)` no escopo global;
- a aplicação é criada apenas dentro da função `create_app()`;
- o módulo `views` é importado normalmente no topo do arquivo;
- o objeto `app` é passado como argumento para outro módulo.

Agora vejamos o segundo arquivo.

Código: views.py

```
1  """Extensao Flask"""
2  def init_app(app):
3      """Inicializacao de extensoes"""
4      @app.route("/")
5      def index():
6          return "Ola Mundo!"
7      @app.route("/contato")
8      def contato():
9          return "<form><input type='text'></form>"
```

E, aqui está a mudança conceitual mais importante:

- Não existe mais `from app import app`;
- O módulo não depende diretamente do objeto global;
- A função `init_app` apenas descreve o que deve ser feito quando receber uma aplicação.

Por Que Isso Resolve o Circular Import? Quando o Python importa o módulo `views`, ele apenas define a função `init_app`. Nenhuma rota é registrada ainda.

Somente quando `create_app()` é executada é que:

1. A instância da aplicação é criada;
2. A função `views.init_app(app)` é chamada;
3. As rotas são registradas.

Ou seja, eliminamos a dependência circular porque:

- `views.py` não precisa importar `app`;
- `app.py` não depende da execução interna de `views`;
- A ligação entre os módulos acontece de forma controlada.

Mudança de Mentalidade

Antes, tínhamos um objeto global rígido, criado imediatamente.

Agora, temos uma função que constrói a aplicação sob demanda.

Essa diferença parece pequena no código, mas é enorme do ponto de vista arquitetural. Passamos de um modelo acoplado para um modelo configurável e extensível.

Mais do que resolver um erro técnico, o padrão *Application Factory* nos ensina a estruturar aplicações Flask de forma previsível, modular e alinhada com boas práticas de engenharia de software.

Benefícios Arquiteturais

Com essa abordagem:

- eliminamos importações circulares;
- reduzimos acoplamento;
- melhoramos a organização modular;
- facilitamos testes automatizados;
- permitimos múltiplas instâncias da aplicação com configurações diferentes.

Mais do que resolver um erro técnico, a *Application Factory* representa uma mudança de mentalidade: a aplicação deixa de ser um objeto global rígido e passa a ser um componente configurável e extensível.

Application Context e Request Context

O Flask utiliza dois conceitos fundamentais para gerenciar o estado da aplicação durante sua execução:

- Application Context;
- Request Context.

Compreender esses contextos é essencial para evitar erros como acesso indevido a variáveis globais ou uso incorreto de recursos da aplicação.

Application Context

O *Application Context* armazena informações globais da aplicação ativa, como configurações e extensões. Ele permite acessar objetos como `current_app` sem precisar importar diretamente a instância da aplicação.

Isso reduz o acoplamento entre módulos e elimina a necessidade de dependências circulares.

Request Context

O *Request Context* está associado ao ciclo de vida de uma requisição HTTP. Ele contém dados específicos da requisição, como:

- Cabeçalhos;
- Parâmetros;
- Sessão;
- Objeto `request`.

Esse contexto é criado automaticamente pelo Flask a cada requisição e destruído ao final do processamento.

Uso de Blueprints em Arquiteturas Escaláveis

Os *Blueprints* permitem dividir a aplicação em componentes independentes, organizando rotas, controladores e lógica de negócio por domínio funcional.

Benefícios do uso de Blueprints:

- Separação clara de responsabilidades;
- Reutilização de módulos;
- Organização lógica do sistema;
- Facilidade de manutenção e expansão.

O registro de blueprints deve ser feito dentro da função *factory*, garantindo que todos os componentes sejam inicializados corretamente.

Configuração por Ambiente

Aplicações profissionais raramente utilizam uma única configuração. É recomendável separar ambientes como:

- Desenvolvimento (development);
- Testes (testing);
- Produção (production).

Isso pode ser realizado por meio de classes de configuração e variáveis de ambiente, permitindo maior segurança e flexibilidade operacional.

Boas Práticas de Arquitetura em Projetos Flask

Para garantir a qualidade arquitetural de aplicações Flask de médio e grande porte, recomenda-se:

- Utilizar o padrão Application Factory;
- Evitar variáveis globais desnecessárias;
- Centralizar a configuração da aplicação;
- Desacoplar extensões da instância da aplicação;
- Organizar o projeto em pacotes coesos;
- Adotar princípios de separação de responsabilidades.

Essas práticas contribuem diretamente para a construção de sistemas mais robustos, escaláveis e alinhados com princípios modernos de engenharia de software.

Síntese da Unidade

Nesta unidade, avançamos da construção inicial de aplicações Flask para uma abordagem arquitetural mais madura, baseada no padrão Application Factory e na correta utilização dos contextos internos do framework. Compreendemos como evitar importações circulares, estruturar projetos modulares e inicializar extensões de forma desacoplada.

Esse conjunto de conceitos estabelece a base necessária para o desenvolvimento de aplicações web profissionais, preparadas para testes automatizados, múltiplos ambientes de execução e evolução contínua do software.